Minha Patinha — Gestão de Animais e Ocorrências SOS

Aluna: Ana Carolina Ferreira | **Data:** Outubro / 2026

Repositório GitHub: https://github.com/anacarolinaferreira02/gestao_animais

Deploy em Produção: http://gestao-animais.infinityfreeapp.com

1. Objetivo

O **Minha Patinha** é um sistema web voltado para cidadãos, protetores independentes e abrigos de animais na cidade de Porto Velho (RO). O objetivo primordial da plataforma é agilizar o resgate de animais perdidos e em situação de vulnerabilidade através de chamados georreferenciados (SOS) com coordenadas geográficas exatas e fotos do local.

Além do módulo de socorro imediato, o sistema centraliza e organiza a rotina operacional do abrigo de animais, fornecendo controle cadastral completo, prontuário clínico veterinário (consultas, exames, vacinas, medicações e evolução de peso) e gestão integrada de todo o ciclo de adoção responsável (solicitação, entrevistas de perfil e acompanhamento pós-adoção). Com moderação ativa da equipe antes da publicação no mapa interativo, a plataforma garante a veracidade das informações, combate o abandono e fortalece a rede de proteção animal no município.

2. Usuários

- **Cidadão (Usuário Comum / Adotante):** Pode se cadastrar livremente com nome, e-mail, telefone e senha; efetuar login e logout; registrar chamados SOS de animais perdidos ou encontrados anexando fotos e marcando o ponto exato no mapa; acompanhar o status de moderação de suas ocorrências na aba "Minhas Ocorrências" (com visualização do motivo caso o chamado seja rejeitado); explorar o mapa público interativo de ocorrências aprovadas; navegar pelo catálogo de animais acolhidos pelo abrigo; e submeter solicitações formais de adoção preenchendo o questionário de aptidão.
- **Administrador (Protetor / Equipe do Abrigo):** Possui acesso restrito ao painel administrativo (`/admin`); gerencia a fila de moderação de ocorrências SOS, avaliando as informações e podendo aprovar o chamado (disponibilizando-o imediatamente no mapa público) ou rejeitá-lo informando obrigatoriamente a justificativa; cadastra, edita e gerencia os animais acolhidos pela organização; registra o prontuário de saúde completo (consultas, vacinas, laudos de exames, prescrições e pesagens); gerencia as solicitações de adoção recebidas, agendando entrevistas e aprovando termos; realiza acompanhamentos pós-adoção com fotos do animal no novo lar; e visualiza a relação geral de usuários cadastrados no sistema.

3. Casos de uso principais

- **UC01:** Cadastrar novo usuário comum com criptografia Bcrypt.
- **UC02:** Autenticar usuário (login) e encerrar sessão ativa com segurança (logout).
- **UC03:** Registrar ocorrência SOS de animal perdido ou encontrado com coordenadas de latitude/longitude e foto.
- **UC04:** Consultar minhas ocorrências e acompanhar status de análise e eventuais justificativas de recusa.
- **UC05:** Visualizar mapa interativo público com marcadores georreferenciados de ocorrências aprovadas.

- **UC06:** Avaliar ocorrência pendente na moderação (aprovar para o mapa ou rejeitar com justificativa obrigatória).
- **UC07:** Cadastrar, editar e gerenciar fichas de animais acolhidos no abrigo com galeria de fotos.
- **UC08:** Registrar prontuário clínico individual (consultas veterinárias, laudos de exames, prescrições e alergias).
- **UC09:** Controlar carteira de vacinação com registro de imunização e cálculo de data da próxima dose.
- **UC10:** Registrar evolução de pesagens periódicas para acompanhamento da saúde do animal.
- **UC11:** Consultar catálogo de animais disponíveis para adoção e submeter formulário de interesse.
- **UC12:** Analisar pedidos de adoção, agendar entrevistas de perfil e registrar termos de aprovação/reprovação.
- **UC13:** Realizar acompanhamento pós-adoção com registro de situação e fotos de vistoria no novo lar.
- **UC14:** Consultar listagem geral de usuários cadastrados na plataforma.

4. Requisitos funcionais

- **RF01:** O sistema deve permitir o cadastro de novos usuários informando nome, e-mail único, telefone e senha de acesso.
- **RF02:** O sistema deve autenticar usuários por meio de e-mail e senha, direcionando para o Dashboard correspondente ao seu nível de acesso (`usuario` ou `admin`).
- **RF03:** O sistema deve permitir encerramento seguro da sessão ativa (logout), limpando todos os dados de sessão do servidor.
- **RF04:** O sistema deve permitir que usuários autenticados registrem ocorrências SOS (tipo "Perdido" ou "Encontrado"), informando características do animal, data, horário, telefone de contato, foto obrigatória e coordenadas de latitude e longitude no mapa interativo.
- **RF05:** O sistema deve salvar novas ocorrências com status inicial "Pendente", impedindo exibição prévia no mapa público.
- **RF06:** O sistema deve permitir que o cidadão consulte suas próprias ocorrências com status em tempo real e visualização da justificativa em caso de rejeição.
- **RF07:** O sistema deve exibir no mapa interativo público apenas as ocorrências com status "Aprovada", utilizando marcadores visuais diferenciados por categoria (Perdido ou Encontrado).
- **RF08:** O sistema deve disponibilizar ao administrador uma fila de moderação para aprovar chamados ou rejeitá-los com preenchimento compulsório de justificativa.
- **RF09:** O sistema deve permitir ao administrador cadastrar animais acolhidos, informando espécie, raça, sexo, idade, porte, cor, peso, dados de resgate, foto e status situacional.
- **RF10:** O sistema deve permitir o registro de prontuário clínico individual: consultas veterinárias, exames com laudo, vacinas com próxima dose, medicamentos com posologia e histórico de alergias.
- **RF11:** O sistema deve manter histórico de pesagens periódicas e galeria de fotos adicionais de cada animal.
- **RF12:** O sistema deve permitir que cidadãos autenticados submetam pedidos de adoção respondendo questionário sobre tipo de moradia e outros animais.
- **RF13:** O sistema deve permitir o gerenciamento do fluxo de adoção (análise de proposta, agendamento de entrevistas e aprovação).
- **RF14:** O sistema deve permitir o registro de visitas e acompanhamentos pós-adoção com data, situação descritiva e fotos no novo lar.
- **RF15:** O sistema deve emitir notificações internas sobre alterações de status em chamados SOS e processos de adoção.

- **RF16:** O sistema deve disponibilizar ao administrador a relação de todos os usuários cadastrados.

5. Requisitos não funcionais

- **Segurança:** Senhas armazenadas obrigatoriamente com hash criptográfico seguro (Bcrypt), nunca em texto puro; todas as consultas implementadas via PDO Prepared Statements prevenindo SQL Injection; isolamento total de credenciais através de variáveis de ambiente sem dados sensíveis no repositório GitHub; controle de acesso nas rotas restritas através de validação de sessão (`$_SESSION["usuario_tipo"] === 'admin'`).
- **Desempenho e Usabilidade:** Interface Mobile-First totalmente responsiva com suporte a toques em smartphones para seleção de pontos no mapa Leaflet; upload de imagens limitado a 5 MB (JPEG, PNG, WebP) com geração de nomes randômicos únicos no servidor para evitar colisões e sobrescritas.

6. Modelo de dados (DER simples)

O banco de dados relacional MySQL é estruturado em 19 tabelas organizadas em 5 módulos de negócio integrados por chaves primárias e estrangeiras:

Módulo 1: Usuários e Notificações

- **usuarios:** id (int, PK, AI), nome (varchar 100), email (varchar 150, Unique), senha (varchar 255), telefone (varchar 20), tipo (enum 'admin', 'usuario'), ativo (tinyint 1), criado_em (timestamp).
- **notificacoes:** id (int, PK, AI), usuario_id (int, FK usuarios.id), titulo (varchar 150), mensagem (text), lida (tinyint 1), criada_em (timestamp).

Módulo 2: Animais e Identificação

- **animais:** id (int, PK, AI), nome (varchar 100), especie (varchar 50), raca (varchar 50), sexo (enum 'Macho', 'Femea'), idade (int), peso (decimal 5,2), cor (varchar 50), porte (enum 'Pequeno', 'Medio', 'Grande'), data_resgate (date), local_resgate (varchar 255), foto (varchar 255), status (enum 'Disponível', 'Em tratamento', 'Em processo de adoção', 'Adotado', 'Perdido', 'Encontrado'), descricao (text), criado_em (timestamp), atualizado_em (timestamp).
- **fotos_animais:** id (int, PK, AI), animal_id (int, FK animais.id), arquivo (varchar 255), legenda (varchar 150), enviada_em (timestamp).
- **historico_status_animal:** id (int, PK, AI), animal_id (int, FK animais.id), status_anterior (varchar 50), status_novo (varchar 50), observacao (text), usuario_id (int, FK usuarios.id), alterado_em (timestamp).

Módulo 3: Ocorrências e SOS (Resgate Georreferenciado)

- **ocorrencias_sos:** id (int, PK, AI), animal_id (int, FK animais.id, Opcional), usuario_id (int, FK usuarios.id), tipo (enum 'Perdido', 'Encontrado'), nome_animal (varchar 100), especie (varchar 50), raca (varchar 50), sexo (enum 'Macho', 'Femea', 'Nao identificado'), cor (varchar 50), porte (enum 'Pequeno', 'Medio', 'Grande'), foto (varchar 255), data_ocorrencia (date), hora_ocorrencia (time), descricao (text), local_desaparecimento (varchar 255), latitude (decimal 10,8), longitude (decimal 11,8), telefone_contato (varchar 20), status (enum 'Pendente', 'Aprovada', 'Rejeitada', 'Resolvido'), motivo_rejeicao (text), moderada_por (int, FK usuarios.id), moderada_em (datetime), criado_em (timestamp), atualizado_em (timestamp).
- **historico_status_sos:** id (int, PK, AI), ocorrencia_id (int, FK ocorrencias_sos.id), usuario_id (int, FK usuarios.id), status_anterior (varchar 50), status_novo (varchar 50), observacao (text), alterado_em (timestamp).

Módulo 4: Prontuário Clínico Veterinário

- **consultas:** id (int, PK, AI), animal_id (int, FK animais.id), veterinario (varchar 150), data_consulta (date), motivo (varchar 255), diagnostico (text), observacoes (text).
- **exames:** id (int, PK, AI), animal_id (int, FK animais.id), tipo (varchar 100), data_exame (date), resultado (text), veterinario (varchar 150), arquivo (varchar 255).

- **vacinas:** id (int, PK, AI), animal_id (int, FK animais.id), nome_vacina (varchar 150), data_aplicacao (date), proxima_dose (date), veterinario (varchar 150), observacoes (text).
- **medicamentos:** id (int, PK, AI), animal_id (int, FK animais.id), nome (varchar 150), dosagem (varchar 100), frequencia (varchar 100), data_inicio (date), data_fim (date), veterinario (varchar 150), observacoes (text).
- **alergias:** id (int, PK, AI), animal_id (int, FK animais.id), alergia (varchar 150), descricao (text).
- **pesos_animais:** id (int, PK, AI), animal_id (int, FK animais.id), peso (decimal 5,2), data_registro (date), observacao (varchar 255).
- **historico_medico:** id (int, PK, AI), animal_id (int, FK animais.id), data_atendimento (date), tipo (varchar 100), descricao (text), veterinario (varchar 150), peso (decimal 5,2), observacao (text), criado_em (timestamp).
- **saude_animais:** id (int, PK, AI), animal_id (int, FK animais.id), tipo (varchar 100), descricao (text), veterinario (varchar 150), data_atendimento (date), proximo_atendimento (date), observacao (text), criado_em (timestamp).

Módulo 5: Adoções e Acompanhamento Pós-Adoção

- **adocoes:** id (int, PK, AI), animal_id (int, FK animais.id), usuario_id (int, FK usuarios.id), nome_completo (varchar 150), cpf (varchar 14), telefone (varchar 20), endereco (varchar 255), tipo_residencia (enum 'Casa', 'Apartamento', 'Chacara/Sitio'), experiencia_animais (tinyint 1), outros_animais (text), status (enum 'Solicitação', 'Em análise', 'Entrevista', 'Aprovada', 'Reprovada', 'Concluída'), observacoes (text), data_solicitacao (datetime), data_conclusao (datetime).
- **entrevistas_adocao:** id (int, PK, AI), adocao_id (int, FK adocoes.id), data_entrevista (datetime), responsavel (varchar 150), resultado (enum 'Aprovado', 'Reprovado', 'Pendente'), observacoes (text).
- **acompanhamentos_adocao:** id (int, PK, AI), adocao_id (int, FK adocoes.id), data_acompanhamento (date), responsavel (varchar 150), situacao (text), observacoes (text), foto (varchar 255).

Relacionamentos estruturais (FK): As tabelas `consultas`, `exames`, `vacinas`, `medicamentos`, `alergias`, `pesos_animais`, `fotos_animais`, `historico_medico` e `saude_animais` referenciam `animais(id)`. A tabela `ocorrencias_sos` referencia `usuarios(id)` e opcionalmente `animais(id)`. A tabela `adocoes` referencia `animais(id)` e `usuarios(id)`. As tabelas `entrevistas_adocao` e `acompanhamentos_adocao` referenciam `adocoes(id)`. As notificações referenciam `usuarios(id)`.

7. Critérios de aceite

CA01 — Autenticação com credenciais válidas:
Dado que o usuário possui conta cadastrada no sistema, quando informa e-mail e senha corretos na tela de login, então o sistema autentica a sessão e abre o Dashboard.

CA02 — Tentativa de login com senha incorreta:
Dado que o usuário informa senha incorreta no login, quando submete o formulário, então o sistema não autentica e exibe mensagem de erro amigável sem expor detalhes internos.

CA03 — Proteção de rota administrativa:
Dado que o usuário comum tenta acessar a URL do painel administrativo (`/admin`), quando a página é carregada, então o sistema bloqueia o acesso e redireciona para a página inicial.

CA04 — Envio de ocorrência SOS com coordenadas:
Dado que o usuário preenche a ocorrência com foto, dados obrigatórios e clica no mapa definindo as coordenadas, quando envia o formulário, então o registro é salvo com status inicial "Pendente" aguardando moderação.

CA05 — Moderação com aprovação:
Dado que o administrador analisa uma ocorrência pendente e clica em aprovar, quando a ação é confirmada, então a ocorrência passa a aparecer imediatamente no mapa público.

CA06 — Moderação com rejeição e justificativa obrigatória:
Dado que o administrador analisa uma ocorrência e informa a justificativa de recusa, quando confirma a rejeição, então o status muda para "Rejeitada", a ocorrência não aparece no mapa público e o motivo fica visível para o autor em "Minhas Ocorrências".

CA07 — Bloqueio de rejeição sem justificativa:
Dado que o administrador tenta rejeitar uma ocorrência sem preencher o motivo, quando tenta enviar, então o sistema impede a submissão e exige o preenchimento da justificativa.

CA08 — Cadastro de animal acolhido:
Dado que o administrador cadastra um animal com dados e foto válidos, quando salva o formulário, então o animal é gravado no banco e passa a constar na lista de animais da organização.

CA09 — Prontuário clínico veterinário:
Dado que o profissional cadastra uma nova vacina ou consulta para um animal, quando submete os dados, então o registro é vinculado ao animal e passa a constar em seu prontuário clínico.

CA10 — Solicitação de adoção responsável:
Dado que um usuário interessado preenche o formulário de adoção de um animal disponível, quando submete a proposta, então o registro é criado na tabela de adoções com status "Solicitação" e a equipe do abrigo é notificada.

CA11 — Acompanhamento pós-adoção:
Dado que a equipe realiza a vistoria pós-adoção, quando preenche a situação e anexa a foto no formulário de acompanhamento, então o registro é vinculado à adoção com sucesso.

8. Decisão de stack, funcionamento prático e roteiro de execução

Stack Adotada: PHP 8 modular com extensão PDO e Prepared Statements no backend, MySQL com integridade referencial no banco de dados, HTML5/CSS3 responsivo e JavaScript nativo no frontend, e biblioteca Leaflet.js v1.9.4 com OpenStreetMap para geolocalização. Repositório versionado: `https://github.com/anacarolinaferreira02/gestao_animais`.

Como o Sistema Funciona na Prática

O sistema atua em duas frentes complementares: o cidadão cadastra animais perdidos ou em risco marcando o ponto exato no mapa Leaflet de Porto Velho, adicionando foto e telefone de contato; a ocorrência cai na fila de moderação do abrigo para validação de veracidade. Uma vez aprovada pela equipe, ela fica pública no mapa interativo com marcadores coloridos para que toda a comunidade possa ajudar no resgate e localização. Simultaneamente, o abrigo gerencia as fichas dos animais acolhidos, mantém prontuário de saúde com vacinas, exames e consultas, e coordena todo o fluxo de adoção, desde o questionário inicial do adotante até visitas periódicas de acompanhamento com fotos no novo lar.

Como o Sistema Roda no Computador (Ambiente Local com XAMPP)

Para executar o sistema localmente no computador exatamente como executado no ambiente de desenvolvimento da aluna:

- **Diretório da Aplicação:** O projeto reside na raiz do servidor web em `C:\xampp\htdocs\gestao_animais`.
- **Inicialização dos Serviços:** Abrir o **XAMPP Control Panel** no Windows e clicar em **Start** nos módulos **Apache** (porta 80) e **MySQL** (porta 3306), aguardando ambos ficarem com status verde.

- **Base de Dados:** Acessar `http://localhost/phpmyadmin`, criar a base de dados com o nome `gestao_animais` (charset `utf8mb4_unicode_ci`) e importar o arquivo `schema.sql` existente no projeto.
- **Configuração de Conexão Local Segura:** O arquivo `config/conexao.php` carrega automaticamente o arquivo `config/local_env.php` antes de instanciar o PDO. No computador local do aluno com senha no MySQL, o arquivo `config/local_env.php` contém:

  ```
  putenv("DB_HOST=localhost");
  putenv("DB_PORT=3306");
  putenv("DB_NAME=gestao_animais");
  putenv("DB_USER=root");
  putenv("DB_PASS=alunoifro"); // ou vazio ("") conforme senha do MySQL local
  ```

  Nota de Segurança: Este arquivo é estritamente ignorado no `.gitignore`, cumprindo a exigência do Anexo B da apostila para que nenhuma credencial seja exposta no repositório GitHub.
- **Navegação e Teste:** Acessar `http://localhost/gestao_animais/index.php` no navegador. O teste direto de integridade da conexão pode ser validado pela URL `http://localhost/gestao_animais/teste.php`.

Como o Sistema Roda na Nuvem (Deploy em Produção no InfinityFree)

Conforme previsto na Fase 6, item 8 da apostila do Prof. Rafael Pitwak, a aplicação foi publicada no provedor gratuito **InfinityFree** no endereço `http://gestao-animais.infinityfreeapp.com`. O banco de dados remoto roda no host `sql311.infinityfree.com` sob o banco `if0_42984854_gestao_animais`, com tabelas importadas pelo script `importar_infinityfree.sql` e credenciais isoladas por variáveis de ambiente, garantindo funcionamento completo em nuvem sem custos.